A Crash Course on Sequential Decision Making, Bellman's Principle, and Bellman's Recursion

(Companion tutorial: qubit flux sensor and radar beam search)

Abstract

This tutorial is a self-contained pedagogical companion to the joint-DP framework of the main paper. It builds the vocabulary of sequential Bayesian decision making from scratch — states, beliefs, actions, policies, rewards — states Bellman's principle of optimality in plain language and in formulas, and works the associated backward-induction recursion in full on two representative problems: the superconducting-qubit Ramsey flux sensor (continuous state, belief compressed to a count-tuple) and the radar beam-search POMDP (discrete state, belief lives natively on a probability simplex). No prior exposure to reinforcement learning or Bellman equations is assumed; basic probability and elementary linear algebra suffice.

Contents

1 What sequential decision making is, and why it needs its own theory

A single Bayesian measurement is a one-shot decision: pick an experimental setting, record a datum, update a posterior, stop. A *sequential* measurement is a cascade of such decisions, with the crucial feature that each action is chosen *after* seeing the outcomes of all previous ones. This freedom to condition is what adaptive sensing trades on: a radar that scans slowly and widely is worth less than one that points at where the previous hit suggested a target might be; a quantum sensor that always uses the same Ramsey delay wastes information that a shorter first probe could have unlocked.

The mathematical awkwardness is that the optimal next action depends on whatever the posterior happens to look like when it is time to pick it — and the posterior depends on every previous choice and every previous outcome, which are themselves random. You cannot just "optimize over action sequences" the way you optimize over a single experimental setting, because an action sequence that performs well against one realization of the data performs poorly against another. The right object to optimize is a *policy*: a rule that maps whatever you currently believe to whatever you should do next. Policies live in a much larger space than action sequences, and the infrastructure for finding the best one is dynamic programming — the subject of this tutorial.

Two running examples carry the load. The first is a superconducting-qubit Ramsey flux sensor: the unknown is a continuous scalar (the external flux), each action is a pair (Ramsey delay, repetition count), and each outcome is a Binomial count. The second is a radar beam-search problem: the unknown is which of sixteen cells on a ring contains a target, each action is a beam pointing, and each outcome is a binary detection. Both have a four-step measurement horizon. The first forces

us to confront a continuous-belief-space obstacle and solve it with a sufficient-statistic reduction; the second is natively finite and lets us see the DP in its rawest form.

2 The building blocks

2.1 The hidden state and the prior

Nature draws a value $\boldsymbol{x}$ from a known prior $p_0(\boldsymbol{x})$. This value is fixed for the duration of the experiment but unknown to us. In the qubit problem $\boldsymbol{x} = \Phi_{\text{ext}}$, a real number in some bounded interval; in the radar problem $\boldsymbol{x} \in \{1, 2, \ldots, 16\}$, a discrete label for which cell the target is in.

2.2 Actions, observations, and the likelihood

At each of K epochs we choose an action $\boldsymbol{s}_k$ from an action set $\mathcal{S}$ and receive an observation $\boldsymbol{y}_k$ drawn from the likelihood

$$\boldsymbol{y}_k \sim p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c}), \tag{1}$$

where $\boldsymbol{c}$ is the physical hardware that parametrizes the likelihood. In the qubit problem the action is (τ, n) — a Ramsey delay and a repetition count — and the observation is the number of "1" outcomes, distributed as $\text{Binomial}(n, p(\boldsymbol{x}, \tau; \boldsymbol{c}))$. In the radar problem the action is a cell to point the beam at and the observation is a bit: 0 for miss, 1 for detection.

2.3 The belief and its update

After observing k epochs of data we know more about $\boldsymbol{x}$ than the prior specified. Our state of knowledge is the posterior

$$\boldsymbol{b}_k(\boldsymbol{x}) \;=\; p(\boldsymbol{x} \mid \boldsymbol{s}_{1:k}, \boldsymbol{y}_{1:k};\, \boldsymbol{c}), \tag{2}$$

called the *belief* at step k. It is obtained from the prior by recursively applying Bayes' rule: $\boldsymbol{b}_0 = p_0$, and after each new measurement

$$\boldsymbol{b}_k(\boldsymbol{x}) \;=\; \frac{p(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c})\, \boldsymbol{b}_{k-1}(\boldsymbol{x})}{\int p(\boldsymbol{y}_k \mid \boldsymbol{x}', \boldsymbol{s}_k;\, \boldsymbol{c})\, \boldsymbol{b}_{k-1}(\boldsymbol{x}')\, d\boldsymbol{x}'}. \tag{3}$$

We denote the mapping defined by Equation (3) by T: $\boldsymbol{b}_k = T(\boldsymbol{b}_{k-1}, \boldsymbol{s}_k, \boldsymbol{y}_k;\, \boldsymbol{c})$. Two facts about the belief that we will use repeatedly:

1. The belief is a sufficient statistic of the history for inference about $\boldsymbol{x}$. Any expectation involving $\boldsymbol{x}$ and conditioning on the history only needs $\boldsymbol{b}_k$, not the raw $(\boldsymbol{s}_{1:k}, \boldsymbol{y}_{1:k})$.
2. The belief is the natural state variable for the optimization problem. A well-posed policy is a function of the current belief, $\boldsymbol{s}_k = \pi(\boldsymbol{b}_{k-1})$, rather than of the raw history.

2.4 The reward and the objective

We need a scalar criterion to rank policies. A pointwise reward $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K};\, \boldsymbol{c})$ is a function that returns a number for each realization of the unknown state, the action sequence, and the hardware. In the qubit problem we use the negative terminal posterior variance (the Bayesian MSE of the posterior-mean estimator is its prior average); in the radar problem we use the mutual information, equivalent to the negative terminal posterior entropy.

The *value* of a policy π is the expected reward when $\boldsymbol{x}$ is drawn from the prior, actions are chosen by π, and observations are sampled from the likelihood:

$$V(\pi) \;=\; \mathbb{E}_{\boldsymbol{x} \sim p_0}\, \mathbb{E}_{\boldsymbol{y}_{1:K} \mid \boldsymbol{x}, \pi, \boldsymbol{c}}\big[\Phi(\boldsymbol{x}, \boldsymbol{s}^{\pi}_{1:K};\, \boldsymbol{c})\big]. \tag{4}$$

The superscript on $\boldsymbol{s}^\pi$ is a reminder that the action sequence is not fixed — each $\boldsymbol{s}_k$ is produced by applying π to the current belief, which depends on the random trajectory $(\boldsymbol{y}_{1:k-1})$.

The *optimal policy* is any $\pi^\star$ that maximizes $V(\pi)$. Solving for $\pi^\star$ is the central task of sequential decision theory.

2.5 Why we cannot just optimize over action sequences

If the horizon were $K = 1$ the optimal policy would reduce to a single best action $\boldsymbol{s}^\star = \arg\max_{\boldsymbol{s}} \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}}[\Phi(\boldsymbol{x}, \boldsymbol{s}; \boldsymbol{c})]$ — ordinary Bayesian optimal experimental design. For $K \geq 2$ two obstructions appear. First, the optimal $\boldsymbol{s}_2$ depends on $\boldsymbol{y}_1$, which is random. So there is no single "best $\boldsymbol{s}_2$"; the best choice is a *function* of $\boldsymbol{y}_1$. Second, the optimal $\boldsymbol{s}_1$ should anticipate how its outcome will shape the best $\boldsymbol{s}_2$ — in other words, $\boldsymbol{s}_1$ must account for the option value of information. Optimizing each epoch separately in a greedy or myopic fashion misses both of these couplings.

The language that lets us talk about this coupling is Bellman's.

3 Bellman's principle of optimality

3.1 The principle in plain language

An optimal policy has the following property. If you follow it up to some moment, reach whatever belief you happen to reach, and then ask "what is the best I can still do from here?" — the continuation of the optimal policy from that belief onward is itself an optimal policy for the sub-problem starting at that belief. Nothing the first part of the policy did will have rendered the second part suboptimal.

That is almost circular, but the content is sharp: the globally optimal policy is *recursive*. You can assemble it by solving the sub-problems (one step before the end, then two before, then three, and so on) and gluing their optimal pieces together.

3.2 The principle in formulas

Fix the hardware $\boldsymbol{c}$. Define the *value-to-go* function

$$V_k(\boldsymbol{b}) \;=\; \max_{\pi} \mathbb{E}\big[\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c}) \,\big|\, \boldsymbol{b}_k = \boldsymbol{b},\ \text{policy } \pi \text{ from step } k \text{ onward}\big], \tag{5}$$

where $\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c})$ is the reward written as a function of the unknown state and the terminal belief (for our two examples Φ_0 is $-(\boldsymbol{x} - \hat{x})^2$ for the qubit, or $\log \boldsymbol{b}_K(\boldsymbol{x}) - \log p_0(\boldsymbol{x})$ for the radar; see Appendix **??**).

$V_k(\boldsymbol{b})$ is the best expected score an optimal agent can still achieve if the current belief is $\boldsymbol{b}$ and there are $K - k$ measurements still to take. Bellman's principle says that this function satisfies the recursion

$$V_K(\boldsymbol{b}) \;=\; \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}}\big[\Phi_0(\boldsymbol{x}, \boldsymbol{b}; \boldsymbol{c})\big], \tag{6}$$

$$V_k(\boldsymbol{b}) \;=\; \max_{\boldsymbol{s}\in\mathcal{S}} \mathbb{E}_{\boldsymbol{y}|\boldsymbol{b},\boldsymbol{s},\boldsymbol{c}}\big[V_{k+1}\big(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c})\big)\big], \quad k = K-1, \ldots, 0. \tag{7}$$

Equation (6) is the *terminal condition*: at step K the experiment is over and the value-to-go collapses to the reward evaluated at whatever belief we have arrived at. Equation (7) is the *Bellman backup*: to know the value-to-go at belief $\boldsymbol{b}$ with $K - k$ steps left, try every action, for each one compute the expected continuation value (which is V_{k+1} evaluated at the next belief, averaged

over the observation that the action would produce), and take the best. The argmax is the optimal action at belief $\boldsymbol{b}$ with $K-k$ steps left:

$$\pi_k^\star(\boldsymbol{b}) \;=\; \arg\max_{\boldsymbol{s}\in\mathcal{S}} \; \mathbb{E}_{\boldsymbol{y}|\boldsymbol{b},\boldsymbol{s},\boldsymbol{c}}\big[V_{k+1}\big(T(\boldsymbol{b},\boldsymbol{s},\boldsymbol{y};\,\boldsymbol{c})\big)\big]. \tag{8}$$

Reading $\pi_0^\star, \pi_1^\star, \ldots, \pi_{K-1}^\star$ off the recursion gives the globally optimal adaptive policy.

3.3 Deriving the recursion from $V(\pi)$

We stated the recursion above. Now we derive it — starting from Equation (4), the expected reward under a fixed policy — in four steps. Each step isolates one ingredient: change of conditioning, sufficiency of the posterior, trajectory factorization through the belief update, and Bellman's principle applied as an interchange of max and expectation.

Step 1: Condition on the data, not on the state. Equation (4) is a nested expectation, outer over the prior p_0 and inner over the data under π. By the law of total expectation (Fubini plus the tower rule), we may swap these:

$$V(\pi) \;=\; \mathbb{E}_{\boldsymbol{y}_{1:K}|\pi,\,\boldsymbol{c}}\big[\,\mathbb{E}_{\boldsymbol{x}|\boldsymbol{y}_{1:K},\,\boldsymbol{c}}\big[\Phi(\boldsymbol{x},\boldsymbol{s}_{1:K}^{\pi};\,\boldsymbol{c})\big]\,\big]. \tag{9}$$

The outer expectation is now over how the data trajectory plays out under π; the inner expectation, over what $\boldsymbol{x}$ could have been given the observed data.

Step 2: Replace the history-dependent integrand by a belief-dependent one. Two logically separate substitutions happen here, both often compressed into the single phrase "sufficient-statistic reduction." They are worth separating, because the first is automatic and the second is a real assumption on the reward.

(2a) Conditioning change: $p(\boldsymbol{x} \mid \boldsymbol{y}_{1:K}) = \boldsymbol{b}_K$. The posterior $\boldsymbol{b}_K$ is by definition the conditional distribution of $\boldsymbol{x}$ given the measurement history:

$$\boldsymbol{b}_K(\boldsymbol{x}) \;=\; p(\boldsymbol{x} \mid \boldsymbol{y}_{1:K}, \boldsymbol{s}_{1:K};\,\boldsymbol{c}). \tag{10}$$

Consequently, for any integrand $f(\boldsymbol{x})$ — no matter how chosen —

$$\mathbb{E}_{\boldsymbol{x}|\boldsymbol{y}_{1:K},\,\boldsymbol{c}}\big[f(\boldsymbol{x})\big] \;=\; \int f(\boldsymbol{x})\,\boldsymbol{b}_K(\boldsymbol{x})\,d\boldsymbol{x} \;=\; \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}\big[f(\boldsymbol{x})\big]. \tag{11}$$

This is just the definition of conditional expectation. The "history" $\boldsymbol{y}_{1:K}$ and the "belief" $\boldsymbol{b}_K$ are interchangeable as the conditioning object for any function of $\boldsymbol{x}$. Point (2a) is therefore free.

(2b) Function substitution: $\Phi(\boldsymbol{x},\boldsymbol{s}_{1:K};\,\boldsymbol{c}) = \Phi_0(\boldsymbol{x},\boldsymbol{b}_K;\,\boldsymbol{c})$. This is the part that requires care. The original reward Φ takes the action trajectory $\boldsymbol{s}_{1:K}$ as an argument; we want to replace that argument by the terminal belief $\boldsymbol{b}_K$. The substitution is legitimate iff Φ's dependence on $\boldsymbol{s}_{1:K}$ actually factors through $\boldsymbol{b}_K$.

Definition (belief-encodable reward). We call Φ *belief-encodable* if there is a function $\Phi_0(\boldsymbol{x},\boldsymbol{b};\,\boldsymbol{c})$ such that, for every $\boldsymbol{x}$, every $\boldsymbol{c}$, and every pair of trajectories $(\boldsymbol{s}_{1:K},\boldsymbol{y}_{1:K})$ and $(\boldsymbol{s}'_{1:K},\boldsymbol{y}'_{1:K})$ whose terminal beliefs agree ($\boldsymbol{b}_K = \boldsymbol{b}'_K$),

$$\Phi(\boldsymbol{x},\boldsymbol{s}_{1:K};\,\boldsymbol{c}) \;=\; \Phi(\boldsymbol{x},\boldsymbol{s}'_{1:K};\,\boldsymbol{c}) \;=\; \Phi_0(\boldsymbol{x},\boldsymbol{b}_K;\,\boldsymbol{c}). \tag{12}$$

In words: any two histories that land on the same terminal posterior yield the same reward. If Equation (12) holds, Φ_0 is uniquely defined (up to values on unreachable beliefs), and substituting the rewrite into Equation (9) is valid.

Two easy cases. The two rewards used in this tutorial both pass the test immediately, because each is a simple functional of the posterior's shape:

- **Posterior variance ($\Phi_{\mathbf{Var}}$).** $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c}) = -(\boldsymbol{x} - \hat{x}(\boldsymbol{b}_K))^2$, where $\hat{x}(\boldsymbol{b}_K) = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{b}_K]$ is the posterior mean. The only place $\boldsymbol{s}_{1:K}$ enters is through $\boldsymbol{b}_K$, and $\boldsymbol{b}_K$ is itself the object we are substituting in. Belief-encodable with $\Phi_0(\boldsymbol{x}, \boldsymbol{b}; \boldsymbol{c}) = -(\boldsymbol{x} - \hat{x}(\boldsymbol{b}))^2$. The belief-level average $\mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}}[\Phi_0]$ equals $-\mathrm{Var}(\boldsymbol{x} \mid \boldsymbol{b})$ — the familiar Bayesian MSE expression.
- **Mutual information / log-posterior-ratio ($\Phi_{\mathbf{MI}}$).** $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c}) = \log \boldsymbol{b}_K(\boldsymbol{x}) - \log p_0(\boldsymbol{x})$. Same argument: $\boldsymbol{s}_{1:K}$ enters only through $\boldsymbol{b}_K$. Belief-encodable with $\Phi_0(\boldsymbol{x}, \boldsymbol{b}; \boldsymbol{c}) = \log \boldsymbol{b}(\boldsymbol{x}) - \log p_0(\boldsymbol{x})$. The belief-level average is $\mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}}[\Phi_0] = \mathrm{KL}(\boldsymbol{b} \| p_0)$; averaging further under the prior gives the mutual information $I(\boldsymbol{x}; \boldsymbol{y}_{1:K})$.

A case where the test fails. The Fisher-information-aligned reward $\Phi_{\log J}(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c}) = \frac{1}{2} \log\det J_N(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})$ has an explicit $\boldsymbol{s}_{1:K}$ dependence through the accumulated Fisher matrix J_N. Two schedules that reach the same terminal posterior $\boldsymbol{b}_K$ but use different delays can have different J_N values (the matrix accumulates a different sum) and therefore different Φ. Equation (12) fails: same $\boldsymbol{b}_K$, different Φ. $\Phi_{\log J}$ is *not* belief-encodable as stated.

The repair. When Equation (12) fails, redefine the "belief" to carry whatever action-dependent accumulator the reward references — in the Fisher case, augment $\boldsymbol{b} \to \tilde{\boldsymbol{b}} = (\boldsymbol{b}, J)$, so that $\Phi_{\log J}(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})$ becomes a function of $(\boldsymbol{x}, \tilde{\boldsymbol{b}}_K, \boldsymbol{c})$. The belief-encodability condition now holds on the augmented belief by construction. The cost is a larger state space for the DP; the benefit is that Step 2b works again. See §7 for the full treatment.

Combining (2a) and (2b). Whenever the reward is belief-encodable (possibly after augmentation),

$$\mathbb{E}_{\boldsymbol{x}|\boldsymbol{y}_{1:K},\boldsymbol{c}}\big[\Phi(\boldsymbol{x}, \boldsymbol{s}^{\pi}_{1:K}; \boldsymbol{c})\big] \overset{(2b)}{=} \mathbb{E}_{\boldsymbol{x}|\boldsymbol{y}_{1:K},\boldsymbol{c}}\big[\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c})\big] \overset{(2a)}{=} \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}\big[\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c})\big]. \tag{13}$$

The first step replaces the integrand by its belief-indexed form (2b); the second step changes the conditioning object (2a), which is automatic. Plugging into Equation (9) gives

$$V(\pi) \;=\; \mathbb{E}_{\boldsymbol{y}_{1:K}|\pi,\boldsymbol{c}}\big[\, \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}\big[\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c})\big] \,\big]. \tag{14}$$

The inner bracket is a pure functional of $\boldsymbol{b}_K$; write $\Phi_0^{\mathrm{belief}}(\boldsymbol{b}; \boldsymbol{c}) \coloneqq \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}}[\Phi_0(\boldsymbol{x}, \boldsymbol{b}; \boldsymbol{c})]$ to give it a name. The two examples above illustrate the two most common forms of Φ_0^{belief}: the posterior variance and the posterior KL to the prior.

Step 3: Factorize the trajectory expectation through the belief update. Any realized trajectory $(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_K)$ can be sampled one step at a time. Draw $\boldsymbol{y}_1$ from $p(\cdot \mid \boldsymbol{b}_0, \pi(\boldsymbol{b}_0); \boldsymbol{c})$, update the belief via $\boldsymbol{b}_1 = T(\boldsymbol{b}_0, \pi(\boldsymbol{b}_0), \boldsymbol{y}_1; \boldsymbol{c})$, draw $\boldsymbol{y}_2$ from $p(\cdot \mid \boldsymbol{b}_1, \pi(\boldsymbol{b}_1); \boldsymbol{c})$, and so on. The outer expectation in Equation (14) thus factorizes as

$$V(\pi) \;=\; \mathbb{E}_{\boldsymbol{y}_1|\boldsymbol{b}_0,\pi,\boldsymbol{c}}\, \mathbb{E}_{\boldsymbol{y}_2|\boldsymbol{b}_1,\pi,\boldsymbol{c}} \cdots \mathbb{E}_{\boldsymbol{y}_K|\boldsymbol{b}_{K-1},\pi,\boldsymbol{c}}\big[\, \Phi_0^{\mathrm{belief}}(\boldsymbol{b}_K; \boldsymbol{c}) \,\big]. \tag{15}$$

Define the *policy-evaluation* value-to-go under a fixed policy π as

$$U_k^{\pi}(\boldsymbol{b}; \boldsymbol{c}) \;\coloneqq\; \mathbb{E}_{\boldsymbol{y}_{k+1},\ldots,\boldsymbol{y}_K|\boldsymbol{b}_k=\boldsymbol{b},\pi,\boldsymbol{c}}\big[\Phi_0^{\mathrm{belief}}(\boldsymbol{b}_K; \boldsymbol{c})\big]. \tag{16}$$

Equation (15) is exactly $V(\pi) = U_0^{\pi}(p_0; \boldsymbol{c})$. The terminal case of Equation (16) is immediate: with no remaining measurements, the expectation degenerates and

$$U_K^{\pi}(\boldsymbol{b}; \boldsymbol{c}) \;=\; \Phi_0^{\mathrm{belief}}(\boldsymbol{b}; \boldsymbol{c}). \tag{17}$$

Deriving the one-step recursion. We now show how U^{π}_{k-1} is determined by U^{π}_{k}, which is the content of the backward recursion. Start from the definition, Equation (16), at step $k-1$:

$$U^{\pi}_{k-1}(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{y}_k, \boldsymbol{y}_{k+1}, \ldots, \boldsymbol{y}_K \mid \boldsymbol{b}_{k-1} = \boldsymbol{b}, \pi, \boldsymbol{c}}\big[\Phi^{\text{belief}}_0(\boldsymbol{b}_K;\, \boldsymbol{c})\big]. \tag{18}$$

Apply the tower rule to split the expectation over $\boldsymbol{y}_k$ from the expectation over the remaining trajectory:

$$U^{\pi}_{k-1}(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{y}_k \mid \boldsymbol{b}_{k-1} = \boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{c}}\Bigg[\underbrace{\mathbb{E}_{\boldsymbol{y}_{k+1}, \ldots, \boldsymbol{y}_K \mid \boldsymbol{b}_{k-1} = \boldsymbol{b}, \boldsymbol{y}_k, \pi, \boldsymbol{c}}\big[\Phi^{\text{belief}}_0(\boldsymbol{b}_K;\, \boldsymbol{c})\big]}_{\text{inner expectation}}\Bigg]. \tag{19}$$

Now examine the inner expectation. Its conditioning variables are $(\boldsymbol{b}_{k-1} = \boldsymbol{b},\ \boldsymbol{y}_k,\ \pi,\ \boldsymbol{c})$. Given these, the belief at step k is determined:

$$\boldsymbol{b}_k \;=\; T\big(\boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{y}_k;\, \boldsymbol{c}\big). \tag{20}$$

The distribution of the remaining trajectory $(\boldsymbol{y}_{k+1}, \ldots, \boldsymbol{y}_K)$ depends on the past only through the current belief (the *sufficiency of the belief* for the future dynamics, in the Markov-chain sense). So conditioning on $(\boldsymbol{b}_{k-1} = \boldsymbol{b},\ \boldsymbol{y}_k)$ is equivalent — for the purpose of computing expectations of functions of the future — to conditioning on the single number $\boldsymbol{b}_k$ from Equation (20):

$$\mathbb{E}_{\boldsymbol{y}_{k+1}, \ldots, \boldsymbol{y}_K \mid \boldsymbol{b}_{k-1} = \boldsymbol{b}, \boldsymbol{y}_k, \pi, \boldsymbol{c}}\big[\,\cdot\,\big] \;=\; \mathbb{E}_{\boldsymbol{y}_{k+1}, \ldots, \boldsymbol{y}_K \mid \boldsymbol{b}_k = T(\boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{y}_k;\, \boldsymbol{c}), \pi, \boldsymbol{c}}\big[\,\cdot\,\big]. \tag{21}$$

The right-hand side of Equation (21), applied to the integrand $\Phi^{\text{belief}}_0(\boldsymbol{b}_K;\, \boldsymbol{c})$, is exactly the definition of U^{π}_{k}, Equation (16), evaluated at belief $T(\boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{y}_k;\, \boldsymbol{c})$:

$$\mathbb{E}_{\boldsymbol{y}_{k+1}, \ldots, \boldsymbol{y}_K \mid \boldsymbol{b}_k = T(\ldots), \pi, \boldsymbol{c}}\big[\Phi^{\text{belief}}_0(\boldsymbol{b}_K;\, \boldsymbol{c})\big] \;=\; U^{\pi}_{k}\big(T(\boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{y}_k;\, \boldsymbol{c});\, \boldsymbol{c}\big). \tag{22}$$

Substituting Equation (22) back into Equation (19):

$$\boxed{U^{\pi}_{k-1}(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{y}_k \mid \boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{c}}\big[\, U^{\pi}_{k}\big(T(\boldsymbol{b}, \pi(\boldsymbol{b}), \boldsymbol{y}_k;\, \boldsymbol{c});\, \boldsymbol{c}\big)\,\big]} \tag{23}$$

which is the policy-evaluation recursion. Shifting the index by one ($k-1 \to k,\ k \to k+1$) and pairing with Equation (17) gives the full backward recursion for U^{π}.

What each step used. Three ingredients, one per substitution:

1. Equation (19) uses the *tower rule* (iterated expectation): joint expectations factor into nested conditional expectations. Applied to separate "first step" from "rest."
2. Equation (20) uses the *deterministic belief update* T: given the previous belief, action, and outcome, the next belief is fixed by Bayes' rule with no remaining randomness.
3. Equation (21) uses the *Markov property of the belief*: the future trajectory's distribution depends on the past only through the current belief. Every piece of history that is not captured in $\boldsymbol{b}_k$ drops out.

No optimization is used anywhere in this derivation; Equation (23) is a valid identity for every fixed policy π. The backward recursion for U^{π} is therefore the right-hand object to carry into Step 4, where we maximize over π to turn it into the Bellman recursion for V_k.

Step 4: Maximize over π (Bellman's principle). Let $V_k(\boldsymbol{b};\, \boldsymbol{c}) \coloneqq \max_\pi U_k^\pi(\boldsymbol{b};\, \boldsymbol{c})$, the value-to-go under the *optimal* continuation from belief $\boldsymbol{b}$ at step k. The recursion for V_k requires commuting $\max_\pi$ with the one-step $\mathbb{E}_{\boldsymbol{y}_{k+1}}$ in Equation (23), and this commutation is the one delicate move in the whole derivation.

Done naively it is wrong. If π were a single fixed function, $\max_\pi \mathbb{E}_{\boldsymbol{y}}[f(\pi, \boldsymbol{y})]$ and $\mathbb{E}_{\boldsymbol{y}}[\max_\pi f(\pi, \boldsymbol{y})]$ need not agree. The step Bellman relies on is that π is *decomposable*. Split the policy at step k into its action at the current belief and the rest:

$$\pi \;=\; (\boldsymbol{s}_k, \pi'), \qquad \boldsymbol{s}_k = \pi(\boldsymbol{b}), \qquad \pi' = \text{the restriction of } \pi \text{ to steps } k+1, \ldots, K-1. \tag{24}$$

The one-step expectation in Equation (23) depends on $\boldsymbol{s}_k$ (which fixes the measurement distribution) and on π' (which determines $U_{k+1}^{\pi'}$). Maximizing jointly,

$$V_k(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \max_{\boldsymbol{s}_k \in \mathcal{S}} \; \max_{\pi'} \; \mathbb{E}_{\boldsymbol{y}_{k+1} | \boldsymbol{b}, \boldsymbol{s}_k, \boldsymbol{c}}\big[\, U_{k+1}^{\pi'}\big(T(\boldsymbol{b}, \boldsymbol{s}_k, \boldsymbol{y}_{k+1};\, \boldsymbol{c})\big)\,\big]. \tag{25}$$

Now $\max_{\pi'}$ *can* be pushed inside $\mathbb{E}_{\boldsymbol{y}_{k+1}}$: adaptive continuations are by construction history-dependent functions, so the maximizer π' is allowed to depend on the realized $\boldsymbol{y}_{k+1}$. Picking the best π' for each $\boldsymbol{y}_{k+1}$ and then averaging gives the same number as maximizing over π' before averaging. This is the *interchange of max and expectation over history-measurable policies* — it would *fail* if π' were restricted to be non-adaptive. Applying the interchange,

$$V_k(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \max_{\boldsymbol{s}_k} \; \mathbb{E}_{\boldsymbol{y}_{k+1} | \boldsymbol{b}, \boldsymbol{s}_k, \boldsymbol{c}}\big[\, \max_{\pi'} \, U_{k+1}^{\pi'}\big(T(\boldsymbol{b}, \boldsymbol{s}_k, \boldsymbol{y}_{k+1};\, \boldsymbol{c})\big)\,\big], \tag{26}$$

and substituting $\max_{\pi'} U_{k+1}^{\pi'} = V_{k+1}$ gives Equation (7) exactly. The terminal condition Equation (6) is immediate from Equation (17): at the last step there is no further action to take, so $V_K = U_K^\pi$ is independent of π.

Evaluating at $k = 0$, $\boldsymbol{b} = p_0$ recovers $V^\star = \max_\pi V(\pi) = V_0(p_0;\, \boldsymbol{c})$, closing the loop from Equation (4) to Equations (6)–(7).

Where the adaptive-versus-fixed distinction lives. The only non-trivial move in the derivation is the interchange of $\max_{\pi'}$ with $\mathbb{E}_{\boldsymbol{y}_{k+1}}$ in Step 4. That interchange is valid *because* adaptive continuations are allowed to depend on the data. If we restricted π' to be a fixed schedule of actions (non-adaptive), $\max_{\pi'}$ would *not* commute with $\mathbb{E}_{\boldsymbol{y}_{k+1}}$, and the Bellman recursion would not simplify. What one would recover instead is the *fixed-schedule* value V_{fixed}, strictly smaller than V_{adaptive} in general. The gap between the two — the "adaptive gain" that the companion paper's EVPI analysis quantifies — is in a precise sense the value of this interchange being legitimate.

3.4 Why this is called *backward* induction

The only equation in the recursion that contains no unknowns is Equation (6) — the terminal one. There is no future to integrate over; only the present belief and the present reward. Once V_K is a known function, Equation (7) at $k = K-1$ becomes solvable, because its right-hand side only references V_K. That gives V_{K-1}, which feeds V_{K-2}, and so on, back to V_0. The value function is constructed layer by layer, from the end of the experiment to the beginning. This is "backward induction on the horizon."

3.5 Value iteration is a different thing

For completeness: Bellman's recursion as stated here is the *finite-horizon* DP method. The name "value iteration" usually refers to its infinite-horizon cousin, in which one repeatedly applies a Bellman operator to a single function V on beliefs and waits for convergence to a fixed point. Convergence there is by contraction (discount factor < 1). In the finite-horizon case there is no iteration to convergence — the terminal condition anchors the recursion and K backward sweeps suffice exactly. All three case studies in the companion paper are finite-horizon, and so is this tutorial.

3.6 Where the computation goes

In principle, the recursion is a finite calculation: there are K layers and each layer is a max over actions of an expectation over observations. In practice the obstruction is that $\boldsymbol{b}$ is a continuous object (a probability density). You cannot tabulate V_k on the infinite space of possible beliefs. Some reduction is needed:

- A sufficient statistic of $\boldsymbol{b}$ that lives in a finite or bounded space. (The qubit example does this.)
- A finitely parametrized model family for $\boldsymbol{b}$, e.g. Gaussian moments. (Case C of the paper does this; the tutorial does not pursue it.)
- A discrete state space in which $\boldsymbol{b}$ is already a finite vector. (The radar example.)

The two worked examples that follow show the first and third of these in detail.

4 Example 1: Qubit flux sensor (continuous state, count-tuple reduction)

4.1 Problem statement

An external magnetic flux Φ_{ext} threads a SQUID loop. Its unknown value $\boldsymbol{x} = \Phi_{\text{ext}}$ is drawn from a uniform prior on $[0, \phi_{\max}\,\Phi_0]$ with $\phi_{\max} = 0.1$. At each of $K = 4$ epochs we pick a Ramsey delay τ_j from a fixed menu of ten log-spaced delays in $[10, 320]$ ns and a repetition count $n \in \{1, 10\}$, then take n Ramsey shots at that delay. The likelihood for a single shot is

$$p(\boldsymbol{y} = 1 \mid \boldsymbol{x}, \tau;\, \boldsymbol{c}) \;=\; \tfrac{1}{2} + \tfrac{1}{2}\,\mathrm{th}(\boldsymbol{x}, \boldsymbol{c})\, e^{-A(\boldsymbol{x},\boldsymbol{c})\,\tau - B(\boldsymbol{x},\boldsymbol{c})^2\,\tau^2}\, \cos\!\big[\Delta\omega(\boldsymbol{x}, \boldsymbol{c})\,\tau\big], \tag{27}$$

where $\mathrm{th}, A, B, \Delta\omega$ are closed-form expressions in $\boldsymbol{x}$ and the hardware $\boldsymbol{c}$. The outcome of n shots at a fixed delay is $\mathrm{Binomial}(n, p)$. The reward is negative terminal posterior variance, $\Phi_0(\boldsymbol{b}_K) = -\mathrm{Var}(\boldsymbol{x} \mid \boldsymbol{b}_K)$.

4.2 The measurement history reduces to a count-tuple

The raw measurement history after k epochs is the list

$$\mathcal{H}_k \;=\; \big((\tau^{(1)}, y^{(1)}),\ (\tau^{(2)}, y^{(2)}),\ \ldots\big), \tag{28}$$

one entry per individual shot, recording which delay it was taken at and whether it returned a 1. $\mathcal{H}_k$ can be very long (up to forty entries for $K = 4$ at $n = 10$ per epoch) and it contains redundancy. The redundancy comes from two structural facts about the Ramsey likelihood, Equation (27):

1. *Conditional independence.* Given $\boldsymbol{x}$ and the hardware $\boldsymbol{c}$, different shots are independent. The likelihood of the entire history factors into a product over shots.

2. *Conditional identity within a delay.* Given $\boldsymbol{x}$, all shots at the same delay τ_j have the same success probability $p(\boldsymbol{x}, \tau_j;\, \boldsymbol{c})$. They are therefore not merely independent but i.i.d. within each delay bin.

Together these facts let us evaluate the likelihood of $\mathcal{H}_k$ by counting. Let N_j be the cumulative number of shots ever taken at delay τ_j in $\mathcal{H}_k$, and M_j the cumulative number of those shots that returned a 1. Then

$$p(\mathcal{H}_k \mid \boldsymbol{x};\, \boldsymbol{c}) \;=\; \prod_{j=1}^{J} p(\boldsymbol{x}, \tau_j;\, \boldsymbol{c})^{M_j} \left[1 - p(\boldsymbol{x}, \tau_j;\, \boldsymbol{c})\right]^{N_j - M_j}. \tag{29}$$

Every other aspect of the history — the order of the shots, how they were split across epochs, how n was partitioned between 1-shot and 10-shot bursts — drops out of Equation (29).

The pair $(\mathbf{N}, \mathbf{M}) = (N_1, \ldots, N_J,\; M_1, \ldots, M_J)$ is, by the Fisher–Neyman factorization theorem, a *sufficient statistic* of the history for inference about $\boldsymbol{x}$: any two histories that agree on $(\mathbf{N}, \mathbf{M})$ are indistinguishable from the point of view of $\boldsymbol{x}$. We arrange them into a single $2J$-integer object called the *count-tuple*,

$$\mathbf{n}_k \;=\; \big(N_1, M_1,\; N_2, M_2,\; \ldots,\; N_J, M_J\big). \tag{30}$$

$\mathbf{n}_k$ is lossless (nothing about $\boldsymbol{x}$ has been discarded), permutation-invariant (shot order does not appear), and bounded (each $N_j \le K n_{\max} = 40$).

4.3 The belief reduces automatically

The belief at step k is a functional of the history $\mathcal{H}_k$ through Bayes' rule:

$$\boldsymbol{b}_k(\boldsymbol{x}) \;=\; \frac{p_0(\boldsymbol{x})\, p(\mathcal{H}_k \mid \boldsymbol{x};\, \boldsymbol{c})}{\int p_0(\boldsymbol{x}')\, p(\mathcal{H}_k \mid \boldsymbol{x}';\, \boldsymbol{c})\, d\boldsymbol{x}'}. \tag{31}$$

But the numerator depends on $\mathcal{H}_k$ only through Equation (29), which depends only on $\mathbf{n}_k$. Consequently

$$\boldsymbol{b}_k(\boldsymbol{x}) \;=\; \boldsymbol{b}(\boldsymbol{x} \mid \mathbf{n}_k;\, \boldsymbol{c}) \;\propto\; p_0(\boldsymbol{x}) \prod_{j=1}^{J} p(\boldsymbol{x}, \tau_j;\, \boldsymbol{c})^{M_j} \left[1 - p(\boldsymbol{x}, \tau_j;\, \boldsymbol{c})\right]^{N_j - M_j}. \tag{32}$$

The belief is a deterministic function of the count-tuple: once you know $\mathbf{n}$, you know $\boldsymbol{b}$. Two histories with the same count-tuple have the same belief, regardless of how the measurements were ordered or split.

4.4 The DP table reduces for free

Now the Bellman machinery inherits the reduction directly. V_k is a function of the belief; the belief is a function of the count-tuple; therefore V_k is a function of the count-tuple: $V_k(\boldsymbol{b}) = V_k(\mathbf{n})$. The tabulation space collapses from the infinite-dimensional space of densities on $[0, \phi_{\max}\Phi_0]$ to the space of $2J$-integer tuples with bounded entries.

The chain of reductions is:

$$\underbrace{\mathcal{H}_k}_{\text{raw history, long list}} \;\longrightarrow\; \underbrace{\mathbf{n}_k}_{\text{count-tuple, } 2J \text{ integers}} \;\longrightarrow\; \underbrace{\boldsymbol{b}_k}_{\text{posterior, density}} \;\longrightarrow\; \underbrace{V_k(\mathbf{n}_k)}_{\text{value function, one scalar per tuple}}.$$

The first arrow is the Fisher–Neyman reduction on the likelihood; the second arrow is Bayes' rule; the third arrow is the Bellman recursion. Critically, the first arrow is the *only* reduction; the rest

are functional consequences. This is the right way to think of it — not that the belief miraculously compresses, but that the *data* compresses because of a symmetry of the likelihood, and the belief (being a function of the data) inherits the compression automatically.

4.5 Reachability and the size of the DP table

The count-tuple $\mathbf{n}_k$ is bounded: at $K = 4$ epochs with at most $n = 10$ shots each, the total shot count $\sum_j N_j$ is at most 40 and the tuple is sparse (at most four bins touched, because there are at most four epochs). The set of reachable count-tuples across all four epochs, as enumerated by a breadth-first search from the empty tuple, is approximately 10^7 distinct tuples. Each entry is a 64-bit float (the value-to-go). The table fits in memory.

4.6 Backward induction on count-tuples, in detail

Posterior from a count-tuple. To evaluate any expectation the DP needs, we must be able to turn a count-tuple into a posterior. Discretize $\boldsymbol{x}$ on a grid of $K_\Phi = 256$ points $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(K_\Phi)}$ uniformly spaced across $[0, \phi_{\max}\Phi_0]$. The posterior at the i-th grid point, given count-tuple $\mathbf{n}$, is

$$\boldsymbol{b}(\boldsymbol{x}^{(i)} \mid \mathbf{n}) \;\propto\; p_0(\boldsymbol{x}^{(i)}) \prod_{j=1}^{J} p(\boldsymbol{x}^{(i)}, \tau_j; \boldsymbol{c})^{M_j} \left[1 - p(\boldsymbol{x}^{(i)}, \tau_j; \boldsymbol{c})\right]^{N_j - M_j}, \tag{33}$$

with the constant of proportionality fixed by $\sum_i \boldsymbol{b}(\boldsymbol{x}^{(i)} \mid \mathbf{n}) = 1$. The posterior variance and the outcome probabilities needed below are both computed from Equation (33) on the fly.

Marginal outcome probability. If the current posterior (given $\mathbf{n}$) is $\boldsymbol{b}(\boldsymbol{x} \mid \mathbf{n})$ and we take an action (τ, n), the distribution over the number of "1"s is

$$\Pr(m \mid \mathbf{n}, \tau, n) \;=\; \sum_i \boldsymbol{b}(\boldsymbol{x}^{(i)} \mid \mathbf{n}) \,\mathrm{Binomial}\big(m \,\big|\, n,\; p(\boldsymbol{x}^{(i)}, \tau; \boldsymbol{c})\big). \tag{34}$$

This is the marginal likelihood that will serve as the weighting in the Bellman backup's expectation.

Child tuple. If we take (τ_j, n) and observe m "1" outcomes, the new count-tuple has the same entries as $\mathbf{n}$ except (N_j, M_j) becomes $(N_j + n,\, M_j + m)$. We write this $\mathbf{n}'(\mathbf{n}, j, n, m)$.

Bellman backup. Combining the pieces, the recursion is

$$V_K(\mathbf{n}) \;=\; -\mathrm{Var}\big(\boldsymbol{x} \mid \mathbf{n}\big), \tag{35}$$

$$V_k(\mathbf{n}) \;=\; \max_{\substack{j \in \{1,\ldots,10\} \\ n \in \{1,10\}}} \sum_{m=0}^{n} \Pr(m \mid \mathbf{n}, \tau_j, n)\; V_{k+1}\big(\mathbf{n}'(\mathbf{n}, j, n, m)\big), \tag{36}$$

for $k = K-1, \ldots, 0$. The optimal adaptive policy is $\pi_k^\star(\mathbf{n}) = (\tau_{j^\star}, n^\star)$, the argmax of Equation (36). The joint-DP value is $V^\star = V_0(\mathbf{0})$ evaluated at the all-zero tuple.

4.7 A miniature trace

To make Equations (35)–(36) concrete, shrink the problem: $K = 2$, $J = 2$ delays, $n \in \{1\}$ only. There are four candidate actions total, $(\tau_1, 1), (\tau_2, 1)$ repeated at each epoch; twelve distinct child tuples at epoch 2 (two delays times six possible aggregate counts of zero or one across the two shots — but since $n = 1$ at each step the reachable tuples after one epoch are $\{(1,0,0,0),(1,1,0,0),(0,0,1,0),(0,0,1,1)\}$, four of them, each representing a different shot location and outcome). Starting from the all-zero tuple:

1. *Terminal layer* $k = 2$. For each of the at most $4 + 4 \times 4 = 20$ reachable tuples, compute the posterior variance from Equation (33) and store $V_2(\mathbf{n}) = -\mathrm{Var}$.
2. *Layer* $k = 1$. For each of the four tuples reachable after one epoch, try both remaining actions. For each, compute the two child tuples (outcomes $m = 0$ and $m = 1$), look up their V_2 values, and compute the marginal-outcome-weighted average. Take the best of the two actions.
3. *Root* $k = 0$. Start at $\mathbf{n} = \mathbf{0}$. Try both actions. For each, compute the two child tuples at $k = 1$, look up their V_1, and average. Take the best. The result is $V^\star$ at the initial belief; the argmax tells us the optimal first action.

For $K = 4$, $J = 10$, $n \in \{1, 10\}$ the identical loop runs over $\sim 10^7$ tuples and takes minutes on a single thread (the DP is embarrassingly memoized, each tuple visited once). The logic is unchanged.

4.8 What the count-tuple reduction bought us

Without the reduction, a brute-force playout tree at $K = 4$, $|\mathcal{S}| = 20$, and up to 11 outcomes per action would have $(20 \times 11)^4 \approx 2 \times 10^9$ leaves. With the reduction, a permutation of playouts that produces the same count-tuple collapses to one table entry; the reachable state space shrinks by roughly two orders of magnitude. The exactness of the reduction — no information about $\boldsymbol{x}$ is lost by representing a history as counts — is what makes this a *lossless* speedup rather than an approximation.

5 Example 2: Radar beam search (discrete state, finite-belief DP)

5.1 Problem statement

A target sits in one of sixteen cells arranged on a ring: $\boldsymbol{x} \in \{1, \ldots, 16\}$, with uniform prior $p_0(\boldsymbol{x}) = 1/16$. At each of $K = 4$ epochs we point a beam at some cell $\boldsymbol{s}_k \in \{1, \ldots, 16\}$ and observe $\boldsymbol{y}_k \in \{0, 1\}$ with Bernoulli likelihood

$$p(\boldsymbol{y}_k = 1 \mid \boldsymbol{x}, \boldsymbol{s}_k; \boldsymbol{c}) \;=\; p_{\mathrm{FA}} + (p_{\mathrm{D}} - p_{\mathrm{FA}}) \cdot G_{\boldsymbol{c}(\theta_{\boldsymbol{s}_k} - \varphi_{\boldsymbol{x}})}, \tag{37}$$

with $p_{\mathrm{D}} = 0.9$, $p_{\mathrm{FA}} = 0.05$, cell bearings $\varphi_{\boldsymbol{x}=2\pi(\boldsymbol{x}-1)/16}$, and $G_{\boldsymbol{c}}$ a top-hat beam pattern of integer width m (the hardware design variable). The reward is the mutual information $I(\boldsymbol{x}; \boldsymbol{y}_{1:K})$, which equals the prior entropy minus the expected terminal posterior entropy; maximizing it is equivalent to minimizing the expected terminal posterior entropy.

5.2 The belief is a probability vector

Since $\boldsymbol{x}$ is discrete with $|\mathcal{X}| = 16$, the belief $\boldsymbol{b}_k$ is a probability vector in the 15-simplex. It is already finite-dimensional; no reduction is needed. The Bayes update T is elementary: at belief $\boldsymbol{b}$,

action $\boldsymbol{s}$, outcome $\boldsymbol{y}$,

$$\boldsymbol{b}'(\boldsymbol{x}) \;=\; \frac{p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{s};\, \boldsymbol{c})\, \boldsymbol{b}(\boldsymbol{x})}{\sum_{\boldsymbol{x}'} p(\boldsymbol{y} \mid \boldsymbol{x}', \boldsymbol{s};\, \boldsymbol{c})\, \boldsymbol{b}(\boldsymbol{x}')}. \tag{38}$$

The DP is now a straight implementation of Equations (6)–(7) on beliefs.

5.3 Reachable beliefs and the DP table

Here the tabulation strategy is not to enumerate all probability vectors (uncountably many) but to enumerate the *reachable* beliefs — those that can actually arise from the uniform prior by a sequence of allowed actions and allowed outcomes. At $K = 4$ with binary outcomes and $|\mathcal{S}| = 16$, the number of reachable beliefs across all four epochs is bounded by $16^4 \times 2^4 = 10^6$; in practice many fewer because many action-outcome sequences lead to the same belief. A breadth-first traversal from $\boldsymbol{b}_0 = p_0$ records every reachable belief once and assigns it an index in a table.

Alternatively, in the special case of the top-hat beam, the belief at any point has at most a constant number of distinct values — the cells inside the beam, the cells outside, conditioned on each detection history — so the reachable belief has a closed-form structure. But conceptually, the DP is just "belief in, value out."

5.4 Backward induction on beliefs, in detail

Let $\mathcal{B}_k$ be the set of beliefs reachable exactly at step k (so $\mathcal{B}_0 = \{p_0\}$).

Terminal layer. For every $\boldsymbol{b} \in \mathcal{B}_K$,

$$V_K(\boldsymbol{b}) \;=\; \sum_{\boldsymbol{x}=1}^{16} \boldsymbol{b}(\boldsymbol{x}) \log \boldsymbol{b}(\boldsymbol{x}) - (\text{prior entropy constant}), \tag{39}$$

the negative posterior entropy (plus a constant that cancels in the argmax). This is the reward for that terminal belief.

Inductive step. For $k = K-1, K-2, \ldots, 0$ and each $\boldsymbol{b} \in \mathcal{B}_k$:

$$V_k(\boldsymbol{b}) \;=\; \max_{\boldsymbol{s} \in \{1,\ldots,16\}} \sum_{\boldsymbol{y} \in \{0,1\}} \Pr(\boldsymbol{y} \mid \boldsymbol{b}, \boldsymbol{s};\, \boldsymbol{c})\; V_{k+1}\big(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y};\, \boldsymbol{c})\big), \tag{40}$$

where $\Pr(\boldsymbol{y} \mid \boldsymbol{b}, \boldsymbol{s};\, \boldsymbol{c}) = \sum_{\boldsymbol{x}\boldsymbol{b}(\boldsymbol{x})\, p(\boldsymbol{y}|\boldsymbol{x},\boldsymbol{s};\boldsymbol{c})}$ is the marginal outcome probability under the current belief. The argmax is the optimal adaptive action at belief $\boldsymbol{b}$ with $K - k$ steps left.

Root. $V^\star = V_0(p_0)$. The globally optimal policy is the collection $\{\pi_k^\star : \boldsymbol{b} \mapsto \arg\max_{\boldsymbol{s}\ldots}\}$ for $k = 0, \ldots, K-1$.

5.5 A miniature trace: one cell, one beam, $K = 2$

To see the structure of an optimal adaptive radar policy, consider the simplification $K = 2$ with a narrow top-hat beam that covers exactly one cell ($m = 1$). From the uniform prior:

1. At epoch 1, for any choice of pointing cell $\boldsymbol{s}_1$, all 16 cells are equally plausible. The Bellman backup says try every $\boldsymbol{s}_1$; by symmetry they are all equivalent, so pick $\boldsymbol{s}_1 = 1$ without loss of generality.

2. Two outcomes are possible. If $\boldsymbol{y}_1 = 1$, the posterior concentrates on cell 1 up to the false-alarm rate; the remaining cells are weighted by $p_{\mathrm{FA}}/[p_{\mathrm{D}}(1 - p_{\mathrm{FA}}) + \ldots]$. If $\boldsymbol{y}_1 = 0$, the posterior shifts weight slightly away from cell 1.
3. At epoch 2, the Bellman backup tries every $\boldsymbol{s}_2$. After a detection at cell 1, the best $\boldsymbol{s}_2$ is to point away from cell 1 at whichever cell now carries the second-highest belief (the MI gain is maximized by sampling from a cell you are uncertain about). After a miss at cell 1, the best $\boldsymbol{s}_2$ is similarly to point elsewhere.

In words: the argmax at epoch 2 is *different* depending on whether $\boldsymbol{y}_1 = 0$ or $\boldsymbol{y}_1 = 1$. That is the adaptive gain. A fixed schedule of $(\boldsymbol{s}_1 = 1, \boldsymbol{s}_2 = 2)$ commits to cell 2 before seeing $\boldsymbol{y}_1$ and thus cannot exploit the information in the outcome.

For the full $K = 4$, $m = 9$ problem (the in-family optimum of the companion paper's Section 5), the Bellman backup produces an adaptive policy that behaves like binary search on the posterior support: the first beam splits the ring in half; each subsequent beam targets the half whose posterior weight is highest, narrowing by roughly a factor of two per step.

6 What the two examples share, and where they differ

	Qubit (Case B)	**Radar (Case A)**
State space $\mathcal{X}$	continuous, $[0, \phi_{\max}\Phi_0]$	discrete, $\{1, \ldots, 16\}$
Belief object	density on $\mathcal{X}$	probability vector in $\mathbb{R}^{16}$
Belief reduction	count-tuple (20 integers)	(none needed)
Action set $\mathcal{S}$	10 delays $\times$ 2 rep counts	16 pointing cells
Observation $\boldsymbol{y}$	Binomial(n, p)	Bernoulli
Outcome count per action	2 or 11	2
Horizon K	4	4
Reward	$-\mathrm{Var}(\boldsymbol{x} \mid \boldsymbol{b}_K)$	$-H(\boldsymbol{b}_K)$ (equiv. MI)
DP table size	$\sim 10^7$ count-tuples	$\sim 10^6$ reachable beliefs

The commonalities are load-bearing. Both problems fit the POMDP template, both reward families are scalar functionals of the terminal belief, both Bellman recursions take the form of Equation (7), and both are solved exactly by backward induction. The divergences are in the shape of the belief and the size of the DP table, both of which are consequences of the state-space dimensionality.

Two lessons worth taking away:

1. **Reduction is essential in continuous-state problems.** The qubit problem only becomes computable because the Ramsey likelihood has a permutation symmetry that collapses the posterior to a 20-integer tuple. A generic continuous-state POMDP has no such luck and must be relaxed — Gaussian-moment propagation (EKF), particle filters, or neural-belief-state methods — at which point the DP is no longer exact.
2. **Exactness is cheap in discrete-state problems.** The radar problem runs directly on the natural probability-vector belief and returns a provably optimal policy without approximations, as long as the reachable belief set fits in memory. At $K = 4$, $|\mathcal{X}| = 16$ it does; at $K = 20$ or $|\mathcal{X}| = 256$ it would not, and the same relaxation pathway as the qubit problem would be needed.

7 Belief augmentation: when the posterior is not enough

Both examples in this tutorial share a quiet simplification: the reward is a functional of the terminal posterior $\boldsymbol{b}_K$ alone. In the qubit problem $-\mathrm{Var}(\boldsymbol{x} \mid \boldsymbol{b}_K)$ depends only on the shape of the posterior, and in the radar problem so does $-H(\boldsymbol{b}_K)$. Once the posterior is determined, the reward is determined — no reference to which actions produced it. This is what lets Step 2 of §3.3 go through: $\boldsymbol{b}_K$ serves as a sufficient statistic of the trajectory *for the reward*, not just for $\boldsymbol{x}$.

Not every sensing reward has this property. The main paper's Case C uses the Fisher-information-aligned reward

$$\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K};\, \boldsymbol{c}) \;=\; \tfrac{1}{2}\, \log\det J_N(\boldsymbol{x}, \boldsymbol{s}_{1:K};\, \boldsymbol{c}), \tag{41}$$

where the accumulated Fisher-information matrix is

$$J_N(\boldsymbol{x}, \boldsymbol{s}_{1:K};\, \boldsymbol{c}) \;=\; J_0 + \sum_{k=1}^{K} n_k\, J_F(\boldsymbol{x}, \tau_k;\, \boldsymbol{c}), \tag{42}$$

and J_F is the per-shot Fisher matrix. The issue is immediate: J_N depends *explicitly* on the action trajectory $\boldsymbol{s}_{1:K} = ((\tau_1, n_1), \ldots)$, not just on the posterior those actions produced. Two schedules that happen to land on the same $\boldsymbol{b}_K$ via different $\{(\tau_k, n_k)\}$ choices give different J_N, hence different reward.

In other words: for $\Phi_{\log J}$, $\boldsymbol{b}_K$ is *not* a sufficient statistic of the trajectory for the reward. Step 2 of the derivation in §3.3 fails on its face. A direct application of Bellman's recursion on $\boldsymbol{b}_k$ alone would miscompute the value function — it would forget the schedule-dependent accumulator.

7.1 The fix: enlarge what "belief" means

The cure is to redefine the state of the DP so that whatever extra information the reward requires rides along. For $\Phi_{\log J}$ we define the *augmented belief*

$$\tilde{\boldsymbol{b}}_k \;\coloneqq\; \big(\boldsymbol{b}_k,\; J_k\big), \qquad J_k \;\coloneqq\; J_0 + \sum_{j=1}^{k} n_j\, J_F(\boldsymbol{x}, \tau_j;\, \boldsymbol{c}). \tag{43}$$

Two quantities now move together: the posterior over $\boldsymbol{x}$, which still updates by Bayes' rule, and the accumulated Fisher matrix, which updates by additive accumulation:

$$J_{k+1} \;=\; J_k + n_{k+1}\, J_F(\boldsymbol{x}, \tau_{k+1};\, \boldsymbol{c}). \tag{44}$$

The augmented update operator is

$$\tilde{T}\big(\tilde{\boldsymbol{b}}, \boldsymbol{s}, \boldsymbol{y};\, \boldsymbol{c}\big) \;=\; \big(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y};\, \boldsymbol{c}),\; J + n\, J_F(\boldsymbol{x}, \tau;\, \boldsymbol{c})\big), \tag{45}$$

with $\boldsymbol{s} = (\tau, n)$. The reward at the terminal augmented belief is a pure functional of $\tilde{\boldsymbol{b}}_K$:

$$\tilde{\Phi}_0(\tilde{\boldsymbol{b}}_K;\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{x}\mid \boldsymbol{b}_K}\big[\tfrac{1}{2}\, \log\det J_K\big]. \tag{46}$$

With $(\boldsymbol{b}, T, \Phi_0)$ replaced by $(\tilde{\boldsymbol{b}}, \tilde{T}, \tilde{\Phi}_0)$, the four-step derivation of §3.3 goes through unchanged — in particular, $\tilde{\boldsymbol{b}}_K$ *is* a sufficient statistic of the trajectory for the augmented reward, so Step 2 is rescued. Bellman's recursion now lives on $\tilde{\boldsymbol{b}}$, and backward induction produces the correct $V_k(\tilde{\boldsymbol{b}};\, \boldsymbol{c})$.

7.2 What augmentation costs and what it buys

The cost is bookkeeping: for a d-dimensional state $\boldsymbol{x}$, J_k is a symmetric $d \times d$ matrix, carrying $d(d+1)/2$ extra real numbers in the state. In exact DP this multiplies the effective state-space dimensionality and the table size. In moment-based DP (Case C of the paper), where $\boldsymbol{b}_k$ is already a compact Gaussian summary, the augmentation by J_k is a mild expansion of a small tuple.

The benefit is that the Bellman machinery proceeds identically to the unaugmented case. The value function $V_k(\tilde{\boldsymbol{b}}; \boldsymbol{c})$ is well-posed, the recursion is exact, the argmax produces an optimal adaptive policy, and (importantly) the envelope-theorem outer gradient on $\boldsymbol{c}$ that the companion paper exploits applies verbatim to the augmented belief.

7.3 A rule of thumb for when to augment

Look at the pointwise reward. List every action-dependent quantity it references. For each, ask: is it a functional of the posterior $\boldsymbol{b}_K$ alone? If yes, leave the belief unaugmented; if no, augment by whichever accumulator would make the reward a functional of the augmented belief. Common triggers:

- **Fisher-information rewards.** $\log\det J_N$, $\operatorname{tr} J_N$, any function of an action-accumulated Fisher matrix. Augment by J_k.
- **Cumulative cost or budget constraints.** A reward that penalizes $\sum_k c(\boldsymbol{s}_k)$ — shot-count budgets, power budgets, time-resource constraints. Augment by the running cost sum.
- **Path-dependent rewards.** Rewards that integrate observations or actions over the trajectory (entropy production, work extraction, dwell-time penalties). Augment by the relevant path integral.
- **Constraint indicators.** "Has the trajectory stayed in a safe set?" Augment by a binary safety flag that latches off the first time the trajectory leaves the set.

The recipe is always the same: redefine the DP's state as (posterior, running accumulator), so that the reward is a functional of the new state and the Bayesian update T is closed over the new state. The tutorial's qubit and radar examples correspond to the trivial case where no accumulator is needed.

8 Connection to joint hardware-policy co-design

Everything in this tutorial so far has assumed the hardware $\boldsymbol{c}$ is fixed: the DP produces the optimal adaptive policy $\pi^\star(\boldsymbol{c})$ for whichever hardware was handed in. The companion paper wraps this inner DP inside an outer optimization over $\boldsymbol{c}$ — gradient descent on

$$V^\star(\boldsymbol{c}) \;\coloneqq\; V_0(p_0; \boldsymbol{c}). \tag{47}$$

This section describes how the outer gradient $\partial V^\star/\partial \boldsymbol{c}$ is computed, and why — despite the Bellman recursion being a max-of-expectations nested K levels deep — the gradient comes out of ordinary reverse-mode automatic differentiation with a single structural trick.

8.1 Three enabling observations

- **Envelope theorem.** The derivative of $V^\star(\boldsymbol{c})$ at a point where the inner policy is Bellman-optimal does not require differentiating through the argmax. The inner policy is treated as a constant function of $\boldsymbol{c}$ when taking the outer gradient; only the *explicit* $\boldsymbol{c}$-dependence of the reward and the belief update matters.

- **Policy-iteration warm-starting.** As $\boldsymbol{c}$ drifts under outer gradient steps, the previously computed $\pi^\star(\boldsymbol{c}_{\text{old}})$ is near-optimal for $\boldsymbol{c}_{\text{new}}$. A full rebuild of the inner DP is not needed every outer step; a partial update suffices.
- **Decoupling of problem sizes.** The cost of the outer loop scales with d_c (the dimension of $\boldsymbol{c}$); the cost of the inner loop scales with the DP table size. These are largely independent, which is why $d_c \sim 10^5$ (the photonic case) can coexist with inner DPs that fit in memory.

8.2 Differentiating the Bellman recursion through $\boldsymbol{c}$

The central obstruction to a mechanical gradient calculation is the max in the Bellman backup, Equation (7). The max is non-smooth, and a naive differentiation through it would pick up subgradient contributions corresponding to how the argmax itself shifts with $\boldsymbol{c}$. The envelope theorem says those contributions vanish at the optimum. We spell out what remains.

The envelope theorem, formally. Define the "Q-function" at step k,

$$Q_k(\boldsymbol{b}, \boldsymbol{s};\, \boldsymbol{c}) \;\coloneqq\; \mathbb{E}_{\boldsymbol{y}_k|\boldsymbol{b},\boldsymbol{s},\boldsymbol{c}}\big[V_{k+1}(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}_k;\, \boldsymbol{c});\, \boldsymbol{c})\big], \tag{48}$$

so that $V_k(\boldsymbol{b};\, \boldsymbol{c}) = \max_{\boldsymbol{s}_k} Q_k(\boldsymbol{b}, \boldsymbol{s}_k;\, \boldsymbol{c})$ and $\boldsymbol{s}_k^\star(\boldsymbol{b};\, \boldsymbol{c}) = \arg\max_{\boldsymbol{s}_k} Q_k(\boldsymbol{b}, \boldsymbol{s}_k;\, \boldsymbol{c})$. The total derivative of V_k with respect to $\boldsymbol{c}$ splits into an indirect piece (via $\boldsymbol{s}_k^\star$) and a direct piece:

$$\frac{\partial V_k}{\partial \boldsymbol{c}}(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \underbrace{\frac{\partial Q_k}{\partial \boldsymbol{s}_k}(\boldsymbol{b}, \boldsymbol{s}_k^\star;\, \boldsymbol{c}) \cdot \frac{\partial \boldsymbol{s}_k^\star}{\partial \boldsymbol{c}}(\boldsymbol{b};\, \boldsymbol{c})}_{=0 \text{ at the argmax}} \;+\; \frac{\partial Q_k}{\partial \boldsymbol{c}}(\boldsymbol{b}, \boldsymbol{s}_k^\star;\, \boldsymbol{c}). \tag{49}$$

The first term vanishes because $\partial Q_k/\partial \boldsymbol{s}_k = 0$ at an interior argmax (the first-order optimality condition); what survives is just the partial derivative of Q_k with $\boldsymbol{s}_k$ held fixed at $\boldsymbol{s}_k^\star$:

$$\boxed{\frac{\partial V_k}{\partial \boldsymbol{c}}(\boldsymbol{b};\, \boldsymbol{c}) \;=\; \frac{\partial Q_k}{\partial \boldsymbol{c}}(\boldsymbol{b}, \boldsymbol{s}_k^\star;\, \boldsymbol{c}).} \tag{50}$$

Equation (**??**) is the operational form of the envelope theorem: to differentiate V_k with respect to $\boldsymbol{c}$, freeze the argmax and differentiate the integrand. No argmax sensitivity is needed.

Three contributions to $\partial Q_k/\partial \boldsymbol{c}$. Write Q_k as an explicit sum over outcomes (discrete $\boldsymbol{y}_k$ for the qubit and radar problems, integral otherwise):

$$Q_k(\boldsymbol{b}, \boldsymbol{s}_k^\star;\, \boldsymbol{c}) \;=\; \sum_{\boldsymbol{y}_k} \Pr(\boldsymbol{y}_k \mid \boldsymbol{b}, \boldsymbol{s}_k^\star, \boldsymbol{c})\, V_{k+1}\big(T(\boldsymbol{b}, \boldsymbol{s}_k^\star, \boldsymbol{y}_k;\, \boldsymbol{c});\, \boldsymbol{c}\big). \tag{51}$$

Applying the product and chain rules with $\boldsymbol{s}_k^\star$ held fixed,

$$\begin{aligned}\frac{\partial Q_k}{\partial \boldsymbol{c}} \;=\; \sum_{\boldsymbol{y}_k} \Bigg[& \underbrace{\frac{\partial \Pr(\boldsymbol{y}_k \mid \boldsymbol{b}, \boldsymbol{s}_k^\star, \boldsymbol{c})}{\partial \boldsymbol{c}}\, V_{k+1}\big(T(\cdots);\, \boldsymbol{c}\big)}_{\text{(i) outcome-probability term}} \\ & + \Pr(\boldsymbol{y}_k \mid \cdots)\, \underbrace{\frac{\partial V_{k+1}}{\partial \boldsymbol{b}}\big(T;\, \boldsymbol{c}\big) \cdot \frac{\partial T(\boldsymbol{b}, \boldsymbol{s}_k^\star, \boldsymbol{y}_k;\, \boldsymbol{c})}{\partial \boldsymbol{c}}}_{\text{(ii) belief-update term}} \\ & + \Pr(\boldsymbol{y}_k \mid \cdots)\, \underbrace{\frac{\partial V_{k+1}}{\partial \boldsymbol{c}}\big(T;\, \boldsymbol{c}\big)}_{\text{(iii) recursive term}} \Bigg].\end{aligned} \tag{52}$$

Three pieces of the $\boldsymbol{c}$-dependence enter:

1. **Outcome-probability (i).** $\boldsymbol{c}$ changes the likelihood of the measurement $\boldsymbol{y}_k$, which re-weights the future continuations.
2. **Belief-update (ii).** $\boldsymbol{c}$ enters the Bayesian update T through the likelihood factor, so the posterior landed on after observing $\boldsymbol{y}_k$ shifts with $\boldsymbol{c}$.
3. **Recursive (iii).** V_{k+1} itself depends on $\boldsymbol{c}$ — this piece is picked up by the same formula applied at step $k+1$, which is the recursion that gives the gradient its backward-induction structure.

The terminal gradient is $\partial V_K/\partial \boldsymbol{c} = \partial \Phi_0^{\text{belief}}(\boldsymbol{b};\, \boldsymbol{c})/\partial \boldsymbol{c}$, obtained by differentiating the reward at the terminal belief with respect to its own $\boldsymbol{c}$ arguments.

Equation (**??**) mirrors the Bellman recursion itself: same DP graph, same memoization, same backward order. If V fits in memory, so does $\partial V/\partial \boldsymbol{c}$.

8.3 The operational recipe: freeze the argmax, then AD

In practice one does *not* implement Equation (**??**) by hand. Instead the envelope theorem lets reverse-mode AD do the entire calculation in a single sweep, provided the argmax is frozen first. The recipe:

1. **Forward pass (no AD).** Run the backward induction from $k = K$ down to $k = 0$, populating $V_k(\mathbf{n};\, \boldsymbol{c})$ in a memoization dictionary. At each reachable $(k, \mathbf{n})$, *record the argmax action* $\boldsymbol{s}_k^\star(\mathbf{n};\, \boldsymbol{c})$ alongside the value. No AD tape is active during this pass.
2. **Gradient pass (AD-traced).** Re-evaluate $V_0(p_0;\, \boldsymbol{c})$ by traversing the DP graph a second time — but substitute the previously recorded $\boldsymbol{s}_k^\star$ in place of the $\max_{\boldsymbol{s}_k}$. The expression being evaluated is now argmax-free:

$$V_0(p_0;\, \boldsymbol{c}) \;=\; \sum_{\text{paths}} \Big[\prod_{k=1}^{K} \Pr(\boldsymbol{y}_k \mid \boldsymbol{b}_{k-1}, \boldsymbol{s}_k^\star, \boldsymbol{c})\Big]\, \Phi_0^{\text{belief}}(\boldsymbol{b}_K;\, \boldsymbol{c}). \tag{53}$$

   By Equation (**??**) the value computed is numerically identical to the forward-pass value.
3. **Reverse-mode AD.** Wrap Equation (**??**) in `Zygote.pullback` (or an equivalent reverse-mode AD system) and apply the pullback. Because every primitive in the expression is either closed-form in $\boldsymbol{c}$ or $\boldsymbol{c}$-free, AD returns the exact $\partial V_0/\partial \boldsymbol{c}$ without any custom rule.

The essential point: *freezing the argmax turns the Bellman recursion into an ordinary differentiable expression.* Without that step, AD would try to differentiate through the max, producing subgradient terms that the envelope theorem already tells us are zero at the optimum. With that step, the non-smooth max disappears from the AD graph entirely.

8.4 What is $\boldsymbol{c}$-dependent in the qubit problem

To make Equation (**??**) concrete: in the Ramsey flux sensor of §4, only one primitive carries $\boldsymbol{c}$ in the entire DP — the Ramsey single-shot likelihood

$$p(\boldsymbol{x}, \tau;\, \boldsymbol{c}) \;=\; \tfrac{1}{2} + \tfrac{1}{2}\,\mathrm{th}(\boldsymbol{x}, \boldsymbol{c})\, e^{-A(\boldsymbol{x},\boldsymbol{c})\,\tau - B(\boldsymbol{x},\boldsymbol{c})^2\tau^2}\, \cos\big[\Delta\omega(\boldsymbol{x}, \boldsymbol{c})\,\tau\big],$$

whose four rate/coefficient parameters $(\mathrm{th}, A, B, \Delta\omega)$ are closed-form functions of the seven hardware components of $\boldsymbol{c}$. Every other object in the DP inherits its $\boldsymbol{c}$-dependence from this single primitive:

- The marginal outcome probability $\Pr(m \mid \mathbf{n}, \tau, n_{\text{shot}};\, \boldsymbol{c})$ (Equation (34)) is a finite sum of Binomial factors evaluated at $p(\boldsymbol{x}_i, \tau;\, \boldsymbol{c})$; its $\boldsymbol{c}$-gradient comes through p.
- The belief $\boldsymbol{b}(\boldsymbol{x} \mid \mathbf{n};\, \boldsymbol{c})$ (Equation (32)) is a normalized product of Ramsey factors; its $\boldsymbol{c}$-gradient is the log-likelihood-ratio score weighted by the unnormalized posterior and renormalized.

- The terminal reward $V_K(\mathbf{n}; \boldsymbol{c}) = -\mathrm{Var}(\boldsymbol{x} \mid \mathbf{n}; \boldsymbol{c})$ depends on $\boldsymbol{c}$ only through its dependence on $\boldsymbol{b}$.
- The count-tuple state space, the action grid $\{(\tau_j, n_{\text{shot}}) : j, n_{\text{shot}}\}$, and the combinatorial child-tuple update $\mathbf{n} \mapsto \mathbf{n}'$ are $\boldsymbol{c}$-free.

Consequently the AD graph for $\partial V_0/\partial \boldsymbol{c}$ contains a single $\boldsymbol{c}$-dependent primitive (the Ramsey function), composed through the DP tree with the chain rule doing all of the bookkeeping. No custom `rrule` for the Bellman max operator is needed — that is the entire operational payoff of the envelope theorem.

The interested reader should turn to Sections 3 and 4 of the companion paper for the envelope-theorem statement in its full generality and the relaxation hierarchy that kicks in when the inner DP is not tractable even once. This tutorial has covered only the tractable-inner-DP case, which corresponds to the paper's rungs 1 and 2 (Case Studies A and B).

9 Further reading and standard references

- **Classical DP.** Bellman, *Dynamic Programming*, 1957. Bertsekas, *Dynamic Programming and Optimal Control*, volumes I and II, especially the finite-horizon chapters.
- **POMDPs.** Kaelbling, Littman & Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, 1998. Smallwood & Sondik, "The optimal control of partially observable Markov processes over a finite horizon," *Operations Research*, 1973.
- **Bayesian optimal experimental design (one-step).** Chaloner & Verdinelli, "Bayesian experimental design: a review," *Statistical Science*, 1995. Lindley, *Making Decisions*, 1971.
- **Sequential OED.** Huan & Marzouk, "Sequential Bayesian optimal experimental design," *Journal of Computational Physics*, 2016. Kreucher & Kastella, "Sensor management using an active sensing approach," *Signal Processing*, 2005.
- **Reinforcement learning (the large-K, neural-policy regime).** Sutton & Barto, *Reinforcement Learning: An Introduction*, 2018.

A On Φ_0 and the reward conversion from pointwise to belief-level

The pointwise reward $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})$ takes $\boldsymbol{x}$ as an argument; the belief-level reward the DP actually consumes is $\mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}[\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c})]$, which takes only $\boldsymbol{b}_K$ and $\boldsymbol{c}$. The conversion is exactly one integration against the posterior. For the two rewards used in this tutorial:

- **Posterior variance.** $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c}) = -(\boldsymbol{x} - \hat{x}(\boldsymbol{b}_K))^2$ with $\hat{x}(\boldsymbol{b}_K) = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{b}_K]$ the posterior mean. Integrating gives $\mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}[-(\boldsymbol{x} - \hat{x})^2] = -\mathrm{Var}(\boldsymbol{x} \mid \boldsymbol{b}_K)$, a pure functional of $\boldsymbol{b}_K$.
- **Mutual information.** $\Phi_0(\boldsymbol{x}, \boldsymbol{b}_K; \boldsymbol{c}) = \log \boldsymbol{b}_K(\boldsymbol{x}) - \log p_0(\boldsymbol{x})$. Integrating gives $\mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}[\log(\boldsymbol{b}_K/p_0)] = \mathrm{KL}(\boldsymbol{b}_K \| p_0)$. Averaging further over $\boldsymbol{b}_K$ under the prior gives the mutual information $I(\boldsymbol{x}; \boldsymbol{y}_{1:K})$.

A third case (Case C of the companion paper) uses $\Phi_0 = \frac{1}{2} \log\det J_N(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})$, where the Fisher-information matrix J_N depends on the action trajectory explicitly. In that case the "belief" is augmented to carry the accumulated J_N as an extra piece of state, so that the action-dependence folds into the belief and Φ_0 becomes a function of the augmented belief alone.